Problem Description

Although the contents in `airbrakes-mpc-overview` had logical reasoning, they failed to account for an important aspect of quadratic programming: Functions in terms of the variables we wish to minimize must be *linear*. This means our previous cost function

$$J = w_A\left(\hat{h} - h\right)^2 + w_D \sum_{i=1}^{N} \left(\alpha_i - \alpha_{i-1}\right)^2$$

is a valid cost function but not a valid QP problem.

Given a dynamical state $\vec{x}_{i+1} = A\vec{x}_i + B\vec{u}_i$ where $\vec{u}_i$ is external input at timestep i and A and B are matrices, a QP algorithm minimizes

$$\frac{1}{2}\vec{x}^T P\vec{x} + \vec{q}^T\vec{x}$$

subject to

$$l \leq A\vec{x} \leq u$$

where P and A are matrices, $\vec{q}$ is a vector, and l and u are lower and upper bounds respectively.

Solution

To solve this problem, we approximate the function $h(\vec{\alpha})$ with a first-order Taylor series about some known $\vec{\alpha}_0$.

Let $\vec{\alpha}_0$ be some airbrake opening amount guaranteed to reach apogee for the remaining flight time. At the beginning of the flight, it may simply be $\vec{0}$, but as the flight progresses and data becomes clearer, it is likely $\vec{\alpha}_0$ is the previous airbrake opening amounts given by the QP algorithm. To find a Taylor approximation about $\vec{\alpha}_0$, vary each $\left(\vec{\alpha}_0\right)_i$ by a small amount ε of order $0.001 - 0.01$. Then compute the change in h when only one $\left(\vec{\alpha}_0\right)_i$ is modified. Let $\vec{\alpha}_0{}'$ by the new gradient value (recomputed at each i). The value of the gradient at position i is thus

$$\left(\nabla h(\vec{\alpha}_0)\right)_i = \frac{h(\vec{\alpha}_0{}') - h(\vec{\alpha}_0)}{\varepsilon}$$

Repeat by changing each $\left(\vec{\alpha}_0\right)_i$ by ε. Now, the height of the rocket at some $\vec{\alpha}$ close to $\vec{\alpha}_0$ is given by

$$h(\vec{\alpha}) \approx h(\vec{\alpha}_0) + \nabla h(\vec{\alpha}_0)(\vec{\alpha} - \vec{\alpha}_0)$$

In other words, our cost function is now linear: We wish to minimize

$$J = \left(\nabla h(\vec{\alpha}_0)(\vec{\alpha} - \vec{\alpha}_0) + h(\vec{\alpha}_0) - \hat{h}\right)^2 + \text{stickout weight}^2 + \text{smoothness weight}^2$$

Let $\beta = h(\vec{\alpha}_0) - \nabla h(\vec{\alpha}_0)\vec{\alpha}_0 - \hat{h}$ which is constant. Now,

$$J = \left(\nabla h(\vec{\alpha}_0)\vec{\alpha} + \beta\right)^2 + \text{stickout weight}^2 + \text{smoothness weight}^2$$

"stickout weight" wishes to minimize the amount that the airbrakes are out far. This reduces the likelyhood of a plan to have the airbrake open lots for a while and then not a lot. It can be given as

$$\text{stickout weight}^2 = \vec{\alpha}^T S \vec{\alpha}$$

where S is some diagonal matrix of weights for each timeslot. "smoothness weight" minimizes the difference between subsequent timesteps. It uses the Difference matrix, defined as

$$D = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & \\ -1 & 1 & 0 & 0 & 0 & \\ 0 & -1 & 1 & 0 & 0 & \\ 0 & 0 & -1 & 1 & 0 & \\ 0 & 0 & 0 & -1 & 1 & \\ & & & & & \ddots \end{pmatrix}$$

Thus, "smoothness weight" is defined a

$$\text{smoothness weight} = \left(D\vec{\alpha} - \begin{pmatrix} 1 \\ 0 \\ 0 \\ \vdots \end{pmatrix} \left(\vec{\alpha}_{-1}\right)_1 \right)^T Q \left(D\vec{\alpha}_i - \begin{pmatrix} 1 \\ 0 \\ 0 \\ \vdots \end{pmatrix} \left(\vec{\alpha}_{-1}\right)_1 \right)$$

the extra vector term at the end adds the previous airbrake opening value and Q is a diagonal matrix of weights for each timeslot.

Converting to a Matrix Equation

Expanding all expressions in the cost function, we find

$$\text{height term} = \vec{\alpha}^T \nabla h(\vec{\alpha}_0) \nabla^T h(\vec{\alpha}_0) \vec{\alpha} + 2\beta \nabla h(\vec{\alpha}_0) \vec{\alpha}$$

$$\text{stickout weight}^2 = \vec{\alpha}^T S \vec{\alpha}$$

$$\text{smoothness weight} = \vec{\alpha}^T D^T Q D \vec{\alpha}$$

Writing in QP algorithm form, we find

$$P = 2(\nabla h(\vec{\alpha}_0)\nabla^T h(\vec{\alpha}_0) + S + D^T Q D)$$

$$q = 2(\beta \nabla h(\vec{\alpha}_0) - \vec{\alpha}_{-1} D^T Q \begin{pmatrix} 1 \\ 0 \\ 0 \\ \vdots \end{pmatrix}$$

Our constraint matrix A is given by

$$A = \begin{pmatrix} \mathbb{I}_N & 0 \\ 0 & D_N \end{pmatrix}$$

where $\mathbb{I}_N$ and D_n are identity and diagonal matrices of size $\vec{\alpha}$. This allows for constraints on opening angles and angular rates.